

The `tick()` Function

Q47

SVELTE 5 REACTIVITY TICK DOM

INTERVIEW QUESTION

Question. How do you manually tap into the rendering cycle using Svelte's `tick` function, and what does it return?

ANSWER.

****What `tick()` is and what it returns****

- **Returns a Promise:** `tick()` is a function imported from `'svelte'` that returns a **Promise** — a built-in JavaScript object representing a value that will be available later. You `await` it inside an `async` function.
- **Resolves after the flush:** the Promise resolves (becomes ready) **once the current reactive flush finishes** — the flush being Svelte's term for the moment it walks the dirty signals and writes the final values to the DOM (see Q46).
- **No value carried:** `tick()` does **not return a value** when awaited. It is a synchronization primitive — its job is to give you a moment in time, not data.

Why you need it

- **DOM reads are stale mid-handler:** when you write `el.scrollTop = 0` right after a state change, the DOM has **not updated yet** because the flush is scheduled for after your handler returns. Reading `el.scrollHeight` at that instant returns the **old value**.
- **** `await tick()` unblocks the flush: by awaiting the Promise, you yield control back to JavaScript, the microtask queue runs, the flush**

executes and paints**, and execution resumes with the DOM in its new state.

- **The fix is one line:** `count += 1; await tick(); console.log(el.scrollHeight);` — now the log shows the **post-update measurement**.

The rules for correct use

- ****Only inside `async` functions:**** `await` is a JavaScript keyword that only works inside a function declared `async`. Using `await tick()` in a synchronous handler is a syntax error.
- **Use sparingly:** reach for `tick()` only when you genuinely need to **read or measure the DOM** after a state change. Most reactive code never needs it — Svelte's template bindings already reflect new state automatically.
- **Never use it to order side effects:** if you need code to run after state settles, prefer **`$effect`** (see Q20) over manual `tick(). tick()` answers *measurement*, `$effect` answers *reaction*.

App.svelte

```
<script>
  import { tick } from 'svelte';

  let items = $state(['a', 'b']);
  let listEl;

  async function addAndScroll() {
    items.push('c');           // State changed, but DOM is
    not updated yet.
    await tick();              // Wait for the flush to write
    the new <li>.
    listEl.scrollTop = listEl.scrollHeight; // Now measures
    the full list.
  }
</script>

<ul bind:this={listEl}>
  {#each items as item}<li>{item}</li>{/each}
</ul>
```

```
<button onclick={addAndScroll}>add and scroll</button>
```

① **Summary.** `tick()` is imported from `'svelte'` and returns a Promise that resolves after the current reactive flush finishes. `await` it inside an `async` function when you need to read or measure the DOM after a state change; for everything else, prefer `$effect`.

STATE

`items.push('c')`

STATE CHANGED, DOM STALE

YIELDS CONTROL

YIELD POINT

`await tick()`

HANDLER PAUSES, MICROTASK RUNS

FLUSH + RESUME

DOM is current

EXECUTION RESUMES, MEASUREMENTS VALID